Course: Individual Software Development Process' 2026

Iteration Report : Iteration 1

By Ultrasmooth

Chosen Irl Challenge : Project A - Lab Data Management

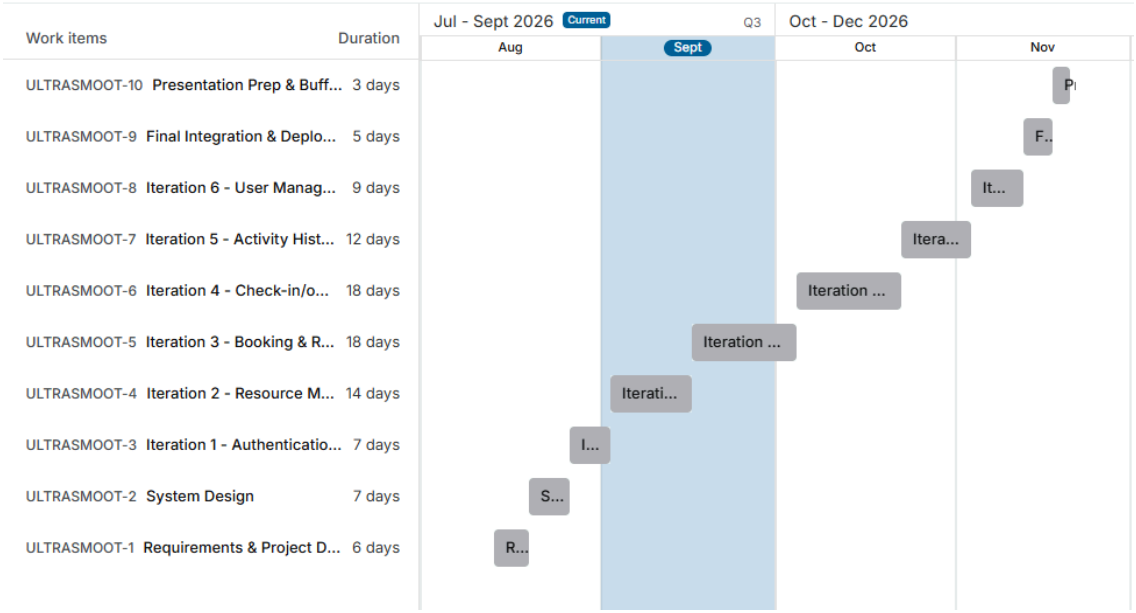
Jirat Kiattrairong	6810545506
Napakhet Namsrioun	6810545727
Panyasiri Aimngern	6810545743
Panisara Niyathirakul	6810545751

Table of Contents

1. Project Snapshot

Project Repository : (WIP)

1.1 Project-Wide Gantt Chart

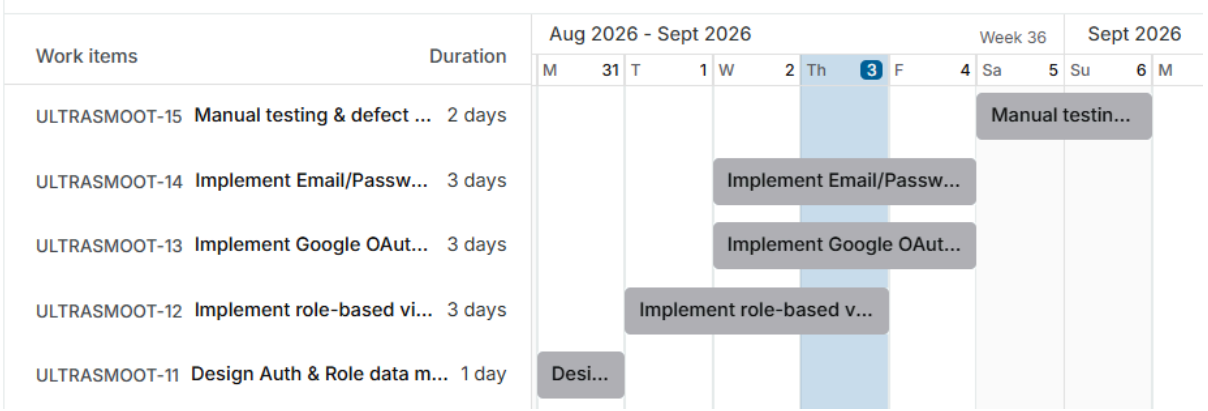


Critical Path :

Requirements & Project Documentation
↓
System Design
↓
Iteration 1 — Authentication & Access (SRS-11, SRS-12)
↓
Iteration 2 — Resource Management (SRS-1, SRS-2)
↓
Iteration 3 — Booking & Request Management (SRS-3, SRS-4, SRS-5)
↓
Iteration 4 — Check-in/Check-out & Maintenance (SRS-10, SRS-6, SRS-7)
↓
Iteration 5 — Activity Log & Dashboard (SRS-8, SRS-9)
↓
Iteration 6 — User Management (SRS-13)
↓
Final Integration & Deployment

Critical Path Duration : 99 calendar days

1.2 Current Iteration Gantt Chart



Critical Path :

Design Auth & Role Data Model

↓

Implement Google OAuth Email/Password Login **(US-11)**

(critical branch - runs in parallel with Implement Role-Based View Logic **(US-12)** but finishes 1 day later)

↓

Manual Testing & Defect Fixes

** Implement Role-Based View Logic (US-12) runs in parallel with the login implementation, starting immediately after the data model and finishing on Sep 3 one day before the OAuth/Email branch. It has 1 day of slack and does not extend the critical path.

Critical Path Duration : 7 calendar days

2. Sprint/Board Status Snapshot

Iteration Goal : In this iteration, we want users to be able to log in safely (with Google or email/password) and only see the parts of the system that match their role. This is the security base that every other feature will be built on top of, so it supports Sub-Goal SG-6 (System Access & Role Management) from our SRS.

Capacity Declared : 2 user stories (we could have done up to 3, but chose to focus on just login and role access). We made this choice because almost every other feature in the system needs a working login system first. If we added a third story about something unrelated, like managing resources, we felt it would split our attention and risk leaving login half-finished and login is the one part where mistakes would cause problems everywhere else later.

User Story	Description	Owner	Status	Within Capacity (Y/N)	Notes
US-11	As a system user, I want to log in via Google OAuth or email/password so I can access the system securely	Napakhet Namsrioun Panisara Niyathirakul	In Progress	Y	This had to be finished first, since nothing else in the system works without a login. Both login methods work and we checked that failed logins show a clear error.
US-12	As a system user, I want role-based access so I can access features according to my role	Jirat Kiattrairong Panyasiri Aimngern	In Progress	Y	Frontend role-based access control has been implemented and manually tested. Users can only see features and pages allowed by their assigned role. Backend authorization and role enforcement are still in progress.

3. Scope Addition/Change Log

Item added/changed	**Reason**	**What as dropped to compensate (one-in-one-out)**
Self-registration (sign-up) entry point for US-11	Needed as a fallback path for users without existing accounts, since Admin-only account creation would block first-time users from accessing the login flow at all	**None**, implemented as a lightweight extension of the existing login UI, not a separate story, so it did not consume additional capacity beyond the declared 2 stories.

4. Retrospective

What went well	**What didn't go well**	**Action items for next sprint**
The login page (frontend) is done and working, including a basic check for the email field	Email validation is still very basic it only checks if the email has an "@" symbol, not if the email and password actually match a real account	Build real validation in Iteration 2: check the email/password against the database, not just the email format
We started early backend scaffolding alongside the frontend, giving us a foundation for Iteration 2 though it's not wired up to the frontend or database yet.	The Google OAuth frontend flow is complete and can be tested (using a mock account chooser). But the real backend connection to Google (callback and token check) is not done yet.	Build the real backend OAuth callback and token check in Iteration 2, since other features depend on users being able to log in

5. Testing & Quality Notes

Note : We did manual testing on the two stories we built this iteration, US-11 (Login/Create Account) and US-12 (Role-based Access). For login, empty fields and invalid emails correctly showed an error, and valid input let the user in (SRS-11). For creating an account, the Student ID field only accepted exactly 10 digits, and the Year field only showed up when the role was Undergraduate, not Ph.D. Student. The role dropdown only lets users sign up as Ph.D. Student or Undergraduate Student, Staff, Admin, and Professor accounts still need to be assigned by an admin, as planned (SRS-11, URS-9). For access control, the "User & Access" menu was hidden for every role except Admin, and trying to open that page directly while not an Admin still got blocked with an error message, so the restriction isn't just a hidden button, it's actually enforced (SRS-12). We also tested Google sign-in with both an existing account (logs in right away) and a new account (asks the user to pick a role first).

No real defects this iteration the tested behavior (validation, error messages, role-based view switching) all runs on the frontend. Backend work so far is limited to early scaffolding and is not yet connected to the frontend or a database, so there wasn't much to break yet. Things like saving data after refresh, real login checks, and full account management (US-13) aren't built yet and will come later. Formal testing against the rest of the SRS, like blocking overlapping bookings (US-3) or missing-field saves (US-2), will start once those features are built.

6. Project Risk Register

Risk ID	Description	Likelihood (L/M/H)	Impact (L/M/H)	Mitigation/ Contingency	Status
R-01	The booking conflict-check logic (SRS-3) might get messier than our current design once we hit real edge cases partial time overlaps, back-to-back bookings, or a rejected/cancelled booking that should free up the slot again	M	H	Nail down all the edge cases before Iteration 2 starts, and write unit tests for the Booking Model's conflict-check before hooking it up to the controller	Monitoring
R-02	With 4 different roles to manage (SRS-12), we could end up leaking access to the wrong role if each controller (Resource, Booking, Maintenance, History & Dashboard) rolls its own permission checks instead of sharing one consistent approach	M	M	Build one shared Auth module/middleware that every controller uses, and test all 4 roles against every endpoint before demo	Open
R-03	Resource status (Available / In Use / Maintenance) could get out of sync if two status-changing actions hit around the same time like a check-out (SRS-10) and a maintenance report (SRS-6) coming in for the same resource almost simultaneously	M	M	Make status updates atomic DB transactions and write out exactly which status transitions are allowed before we start coding	Open

R-04	Nobody on the team has much hands-on experience with Docker Compose running the app + MySQL together, so setup could eat more time than expected once Iteration 2 starts	M	M	Get Docker Compose up and running as its own task early in Iteration 2, separate from feature work, so we've got room to troubleshoot	Open
R-05	Google OAuth 2.0 (SRS-11) depends on an outside service if we misconfigure it, or Google changes something on their end, that whole login path could just stop working	L	H	Build email/password login first so it works on its own no matter what, and treat Google OAuth as an add-on we test separately	Monitoring

7. Individual Contribution Log

Student Name	Tasks owned this sprint	Course concept(s) applied	Evidence	Self-rated effort (1-5)
Napakhet Namsrioun	Built the login page with email/password form, show/hide password functionality, and Google Sign-In flow for US-11. Used mock login validation (valid email and non-empty password) to allow development to continue before the real backend authentication was ready.	Error-state design from lecture that used different messages for empty fields vs. invalid email format, so users know exactly what to fix instead of guessing.	google login photo evidence email login photo evidence	4
Panisara Niyathirakul	Built the create account page for all fields (Student ID, name, faculty, major, email, password), the validation rules, and the logic ensuring only students can create their own accounts.	Following the requirements from SRS-11 and URS-9, which state Admin and Professor accounts cannot be self-created, the role option is limited to students only, with Admin accounts added later by an administrator. This follows the principle of least privilege.	Sign up photo evidence	4
Jirat Kiattrairong	Designed the whole role system with four roles Lab User, Professor, Lab Staff, and Admin decide what each role can see and what actions they can take. Also built the access-control logic ensuring each user can only access the features allowed for their role	Applied to the idea that security should not depend only on hiding buttons or pages in the UI even if a user tries to directly access a restricted page, the system checks their role and blocks them if they don't have permission. Access control is handled in the system logic, not just the interface.	admin photo evidence	4

Panyasiri Aimngern	Wrote the Home and User & Access page content to demonstrate role-based content differences, and tested access control by switching to a non-admin role and trying to open the restricted page. Also tested the completed work from all team members this sprint, scaffolded the initial web application structure, and began early backend implementation.	RBAC applied to page-level access control (testing content differences across roles), and software testing fundamentals from lecture.	role lab users (not admin) role lab admin (with User&Access page) login testing signup testing email validation sign up testing role selection sign up testing studentID	4